



**Politecnico
di Torino**

Politecnico di Torino

Corso di Laurea Triennale in Ingegneria Gestionale L-8

A.A 2025/2026

Sessione di Laurea Luglio 2026

Retailer Affinity Network: Modellazione a grafo delle affinità tra rivenditori e identificazione di cluster forti e deboli per il supporto alle decisioni commerciali

Relatore:

Prof. Giuseppe Bruno Averta

Candidato:

Tommaso Gazzano

<i>Proposta di Progetto.....</i>	<i>4</i>
Studente Proponente	4
Titolo della proposta	4
Descrizione del problema proposto.....	4
Descrizione della rilevanza gestionale del problema.....	4
Descrizione del dataset per la valutazione	5
Descrizione preliminare degli algoritmi coinvolti	5
Descrizione preliminare delle funzionalità previste dall'applicazione Software.....	5
<i>Descrizione dettagliata del problema</i>	<i>7</i>
Obiettivo.....	7
Contesto aziendale.....	7
Input e Output	7
Criticità e potenzialità.....	8
<i>Descrizione del Dataset utilizzato per l'analisi.....</i>	<i>9</i>
<i>Descrizione delle strutture dati e degli algoritmi utilizzati.....</i>	<i>11</i>
Struttura del progetto	11
Package database.....	11
Package model	11
Package UI	12
Algoritmi principali	12
Costruzione del grafo e stampa a video	12
Algoritmi Ricorsivi	13
Visualizzazione del cammino ricorsivo	14
Analisi dei prodotti lungo il cammino	16
<i>Diagramma delle classi dell'applicazione.....</i>	<i>17</i>
<i>Esempio applicativo e risultati sperimentali.....</i>	<i>18</i>
Funzionamento dell'applicazione	18
Esempio Warehouse Store.....	19
<i>Valutazioni sui risultati ottenuti e conclusioni</i>	<i>21</i>

Valutazione del risultati	22
Limiti dell'applicazione	22
<i>Licenza</i>	23

Proposta di Progetto

Studente Proponente

S313657 Tommaso Gazzano

Titolo della proposta

Retailer Affinity Network: Modellazione a grafo delle affinità tra rivenditori e identificazione di cluster forti e deboli per il supporto alle decisioni commerciali.

Descrizione del problema proposto

Nel contesto della distribuzione commerciale, i rivenditori adottano strategie di assortimento molto differenti, che rendono complesso per le aziende produttrici identificare quali partner presentano comportamenti simili o quali competitor operano sugli stessi segmenti di mercato. Questo problema si traduce nella difficoltà di individuare cluster di rivenditori affini, di prevedere potenziali collaborazioni o conflitti e di rilevare punti di debolezza nella rete distributiva. L'obiettivo del progetto è quindi quello di modellare i dati di vendita come un grafo, in cui i collegamenti riflettono la similarità tra i rivenditori, e di esplorare questo grafo per trovare sequenze e catene significative.

Descrizione della rilevanza gestionale del problema

L'analisi proposta ha un forte valore gestionale perché fornisce strumenti per segmentare e comprendere meglio la rete dei rivenditori. Identificare catene di affinità permette a un'azienda di pianificare campagne promozionali mirate e rilevare aree di isolamento commerciale. Inoltre, l'individuazione delle sequenze "deboli" consente di formulare raccomandazioni operative, ad esempio suggerendo un ampliamento dell'assortimento o l'ingresso in nuove categorie di prodotto per rafforzare la posizione di mercato. Si tratta quindi di un supporto decisionale utile per la pianificazione strategica e commerciale.

Descrizione del dataset per la valutazione

L'analisi si basa sull'impiego del database **GOsales**, messo a disposizione durante il corso, il quale contiene informazioni relative ai rivenditori, ai prodotti e alle transazioni effettuate nel tempo, organizzate in diverse tabelle tra loro collegate.

In particolare, dalle tabelle è possibile estrarre, a partire da filtri selezionati dall'utente, l'elenco dei rivenditori attivi e l'insieme dei prodotti da essi commercializzati. Queste informazioni risultano fondamentali per la costruzione del grafo oggetto del progetto.

Il dataset include inoltre una dimensione temporale, rappresentata dalla data delle vendite, che consente di analizzare il comportamento dei rivenditori in specifici periodi e di restringere il dominio di analisi in funzione delle esigenze dell'utente.

Descrizione preliminare degli algoritmi coinvolti

Il problema algoritmico centrale consiste nella ricerca di cammini ottimali in un grafo pesato. A partire da un nodo, rappresentato da un retailer, un algoritmo ricorsivo con backtracking esplora tutte le possibili sequenze di rivenditori collegati, con vincoli sulla lunghezza massima e l'unicità dei nodi. L'obiettivo è identificare i cammini che massimizzano la somma dei pesi, individuando sequenze forti, oppure che minimizzano l'affinità complessiva, individuando sequenze deboli.

Descrizione preliminare delle funzionalità previste dall'applicazione Software

L'applicazione presenta una GUI sviluppata in Python tramite libreria Flet, organizzata secondo il pattern MVC. L'utente ha la possibilità di selezionare dei filtri, quali la tipologia dei rivenditori, periodo temporale e soprattutto soglia di affinità, ovvero il numero minimo di prodotti in comune necessario affinché due rivenditori vengano collegati nel grafo. Questo parametro permette di eliminare collegamenti troppo deboli, come retailer che hanno in comune un solo prodotto, rendendo l'analisi più significativa e concentrata sulle relazioni solide.

Una volta generato il grafo, saranno disponibili tre principali funzionalità:

1. **Analisi della rete:** visualizzazione del grafo dei retailer simili e delle relative statistiche.
2. **Cammino Vincente:** ricerca della sequenza più forte in termini di affinità, utilizzando l'algoritmo ricorsivo di backtracking a partire da un nodo selezionato dall'utente, evidenziando anche quali sono i tre prodotti che si riscontrano maggiormente lungo la sequenza.
3. **Cammino Debole:** individuazione della sequenza più debole, evidenziando i tre prodotti con la frequenza minore, permettendo lo sviluppo di scelte gestionali per rafforzare la posizione dei retailer coinvolti.

Descrizione dettagliata del problema

Obiettivo

L'obiettivo del progetto è lo sviluppo di un modello che consenta di formalizzare la rete commerciale a partire dai dati di vendita, al fine di analizzarne il grado di affinità. In particolare, si costruisce un grafo pesato in cui i nodi rappresentano i retailer e gli archi esprimono la similarità tra di essi, misurata in funzione dei prodotti condivisi.

Su tale struttura vengono applicati algoritmi di ricerca per individuare sequenze di nodi che massimizzano o minimizzano una funzione di peso, permettendo di identificare rispettivamente catene di affinità forti e deboli.

Contesto aziendale

Nel contesto della distribuzione commerciale, le aziende gestiscono reti di rivenditori caratterizzate da comportamenti diversi in termini di assortimento e vendite, rendendo complessa l'analisi delle relazioni tra i vari attori.

L'approccio proposto consente di analizzare tali relazioni in modo quantitativo, offrendo una rappresentazione strutturata della rete commerciale e permettendo di individuare schemi ricorrenti e connessioni significative tra rivenditori.

Input e Output

Il sistema riceve in ingresso dall'utente una serie di parametri di filtro, quali la tipologia di rivenditore, l'intervallo temporale e la soglia minima di affinità. A partire da tali informazioni, viene costruito un grafo in cui i nodi corrispondono ai retailer e gli archi collegano coppie di soggetti che condividono un numero sufficiente di prodotti, mentre il peso di ciascun arco esprime il grado di affinità tra di essi. Su questa struttura vengono applicati algoritmi di ricerca ricorsiva con backtracking, con l'obiettivo di individuare sequenze di nodi che massimizzano o minimizzano il valore complessivo dell'affinità. Il sistema restituisce quindi una rappresentazione della rete e dei cammini individuati, evidenziando le sequenze più forti e quelle più deboli, insieme ai prodotti maggiormente rappresentativi lungo i percorsi analizzati.

Criticità e potenzialità

Una delle principali criticità del modello riguarda la complessità computazionale degli algoritmi ricorsivi utilizzati, che aumenta al crescere del numero di nodi e della lunghezza massima scelta dall'utente. Inoltre, la qualità dell'analisi dipende fortemente dai dati disponibili e dai filtri applicati: soglie di affinità troppo elevate o intervalli temporali troppo ristretti possono ridurre significativamente la densità della rete, limitando il numero di relazioni individuabili.

D'altra parte, il modello presenta diverse potenzialità, in quanto consente di rappresentare in modo strutturato le relazioni tra i rivenditori e di evidenziare schemi non immediatamente osservabili nei dati. La flessibilità dell'approccio permette inoltre di estenderlo introducendo ulteriori vincoli e parametri di analisi. In prospettiva, il modello potrebbe essere integrato con tecniche di machine learning per individuare pattern nascosti o supportare analisi predittive, ampliandone ulteriormente le capacità applicative.

Descrizione del Dataset utilizzato per l'analisi

Il dataset utilizzato è il database relazionale GOsales, fornito durante il corso. Contiene informazioni relative alle vendite, ai prodotti e ai rivenditori. Le relazioni tra queste tabelle consentono di costruire i legami tra i rivenditori e prodotti venduti, costituendo la base per la costruzione del grafo utilizzato nell'analisi.

Sotto viene riportato il diagramma ER corrispondente.

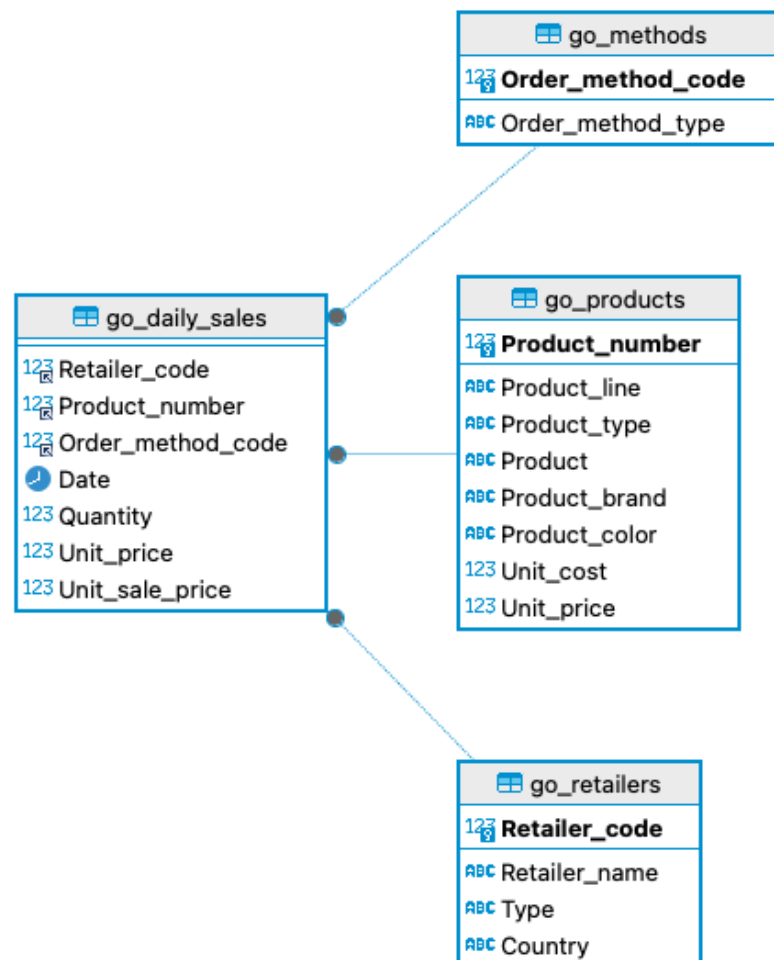


Figura 1: struttura del database GOsales

Per lo sviluppo dell'applicazione sono utilizzate le informazioni contenute nelle seguenti tabelle:

- ***go_daily_sales***: che contiene i seguenti attributi
 - *Retailer_code*: identificativo univoco del retailer (int)
 - *Product_number*: identificativo univoco del prodotto venduto (int)
 - *Date*: data della vendita (datetime)
 - *Quantity*: quantità di prodotto della transazione (int)
 - *Unit_price*: prezzo del singolo prodotto (int)
 - *Unit_sale_price*: prezzo del singolo prodotto quando in sconto (int)

- ***go_products***: che contiene i seguenti attributi
 - *Product_number*: codice identificativo del prodotto (int)
 - *Product_line*: linea del prodotto (string)
 - *Product_type*: tipologia del prodotto (string)
 - *Product*: nome del prodotto (string)
 - *Product_brand*: nome del brand (string)
 - *Product_color*: colore del prodotto (string)
 - *Unit_cost*: costo unitario del prodotto (int)
 - *Unit_price*: prezzo di vendita del prodotto (int)

- ***go_retailers***: che contiene i seguenti attributi
 - *Retailer_code*: codice identificativo del retailer (int)
 - *Retailer_name*: nome del retailer (string)
 - *Type*: tipologia di retailer (string)
 - *Country*: paese in cui si trova il retailer (string)

Descrizione delle strutture dati e degli algoritmi utilizzati

Struttura del progetto

L'applicazione adotta il pattern MVC (Model-View-Controller), modello standard per lo sviluppo di interfacce utente, consentendo la separazione tra logica applicativa, gestione dei dati e interfaccia utente.

Il progetto si divide in tre package differenti.

Package database

All'interno del package database sono contenute le classi necessarie alla gestione delle interazioni tra il server e l'applicazione, suddivise come segue:

- **DB_connect.py**: classe che implementa la connessione al database tramite la libreria mysql-connector-python.
- **DAO.py**: classe che contiene tutti i metodi per estrarre dal database i dati necessari alla costruzione del grafo e l'esecuzione degli algoritmi.

Package model

All'interno del package model si trovano tutte le classi necessarie alla realizzazione della logica applicativa del software. In esso si trovano i file:

- **modello.py**: classe che gestisce la costruzione dei grafi, gli algoritmi ricorsivi e i metodi che permettono di trasformare i grafi creati in un'immagine PNG attraverso l'utilizzo delle librerie NetworkX e Matplotlib.
- **Retailer.py**: "dataclass" utilizzata per modellare i retailer.

Package UI

Il package UI contiene i file dedicati alla gestione dell'interfaccia grafica:

- **view.py**: classe che gestisce la visualizzazione grafica e interattiva dei dati, che consiste nella selezione dei filtri da parte dell'utente e la stampa a video del grafo creato sfruttando la libreria flet.
- **controller.py**: classe che si occupa della validazione degli input, quali tipologia di retailer scelta attraverso un menù a tendina, data di inizio e fine dell'analisi, soglia di affinità, nodo di partenza per gli algoritmi ricorsivi e lunghezza massima dei cammini.

Algoritmi principali

Costruzione del grafo e stampa a video

Una volta selezionati i filtri, alla pressione del tasto “Crea Grafo”, il controller effettua la validazione degli input per evitare incongruenze nei dati inseriti. Successivamente viene avviata la costruzione del grafo attraverso il metodo buildGraph() che si occupa di inizializzare la struttura, aggiungere i nodi che sono stati estratti dal database tramite il DAO e inserire gli archi tra i retailer che soddisfano la soglia di affinità selezionata.

```
def buildGraph(self, rtype, dateFrom, dateTo, affinity): 1 usage new *
    self._graph.clear()
    self.getAllRetailer(rtype, dateFrom, dateTo)
    self._graph.add_nodes_from(self._retailer)
    self.addAllEdges(rtype, dateFrom, dateTo, affinity)

def addAllEdges(self, rtype, dateFrom, dateTo, affinity): 1 usage 2 tommasogazzano
    edges = self.getAllEdges(rtype, dateFrom, dateTo, affinity)
    for r1, r2, w in edges:
        if r1 in self._idMap and r2 in self._idMap:
            n1 = self._idMap[r1]
            n2 = self._idMap[r2]
            self._graph.add_edge(n1, n2, weight=w)
```

Figura 2: creazione del grafo

Una volta creato il grafo, si procede alla generazione e visualizzazione dell'immagine tramite il metodo `drawGraphToFile()`, che si occupa di definire il layout, rappresentare graficamente la struttura e salvarla in un file PNG all'interno della cartella del progetto.

```
def drawGraphToFile(self, outpath: str = "graph.png") -> str: 1 usage  ⚠ tommasogazzano
    """
    Disegna il grafo creato salvandolo in un file png
    :param outpath: percorso del file PNG di output
    :return: percorso del file PNG
    """

    if self._graph.number_of_nodes() == 0:
        raise ValueError("Grafo vuoto, niente da disegnare")

    # rimuove i nodi isolati (senza archi) per evitare di riempire il disegno di
    # informazioni non significative
    isolated_nodes = list(nx.isolates(self._graph))
    self._graph.remove_nodes_from(isolated_nodes)

    if self._graph.number_of_nodes() == 0:
        raise ValueError("Nessun nodo connesso da disegnare")

    pos = nx.circular_layout(self._graph)
    fig, ax = plt.subplots(figsize=(14.4, 10)) #fig = contenitore dell'intera figura (canvas),
    # ax = zona di disegno all'interno di fig
    nx.draw(self._graph, pos=pos, ax=ax, with_labels=False, node_size=300, arrows=False)

    labels = {n: str(n) for n in self._graph.nodes()}
    # sposta leggermente le labels verso l'esterno per evitare sovrapposizione con i nodi
    labels_pos = {n: (pos[n][0] * 1.08, pos[n][1]*1.08) for n in self._graph.nodes()}

    nx.draw_networkx_labels(self._graph, pos=labels_pos, labels=labels, font_size=12, ax=ax)

    edges_labels = nx.get_edge_attributes(self._graph, name='weight')
    nx.draw_networkx_edge_labels(self._graph, pos=pos, ax=ax, edge_labels=edges_labels, font_size=10)

    ax.set_axis_off() #rimuove gli assi cartesiani (default di Matplotlib) per pulire il grafico
    plt.margins(0.20) #aggiunge spazio attorno al grafo per evitare che le labels vengano tagliate
    plt.savefig(*args: outpath, dpi=200, bbox_inches='tight')
    plt.close()

    return os.path.abspath(outpath)
```

Figura 3: Generazione del PNG del grafo

Algoritmi Ricorsivi

L'algoritmo di ricerca ricorsivo implementato nel Model esplora tutte le possibili sequenze di retailer per individuare il cammino ottimale secondo uno dei due criteri,

“cammino vincente” o “cammino debole”. Le possibili soluzioni vengono generate in modo sistematico costruendo progressivamente una serie di percorsi parziali, evitando la ripetizione dei nodi già visitati. Il meccanismo di backtracking, grazie alla rimozione dell’ultimo elemento inserito nel percorso, consente di esplorare tutte le configurazioni possibili.

Il metodo principale avvia la ricerca a partire dal nodo selezionato dall’utente, mentre la funzione ricorsiva costruisce i cammini passo dopo passo: quando viene raggiunta una soluzione completa, il valore del percorso viene confrontato con il migliore trovato fino a quel momento e, in base al criterio selezionato, si aggiorna la soluzione ottimale.

```
def getCamminoVincente(self, retailer_partenza, maxLength): 1 usage  ⚡ tommasogazzano *
    """
    Algoritmo che va a identificare le sequenze forti che vanno a massimizzare
    i pesi degli archi (affinità) tra i nodi (retailer)
    :param retailer_partenza: Nodo di Partenza scelto dall'utente
    :param maxLength: lunghezza massima del cammino
    :return: tupla comprensiva di lista contenente tutti i nodi e valore del peso della sequenza
    """
    self._bestPath = []
    self._bestWeight = 0

    parziale = [retailer_partenza]
    self.ricorsioneCamminoVincente(retailer_partenza, parziale, pesoCorrente=0, maxLength)
    return self._bestPath, self._bestWeight
```

```
def ricorsioneCamminoVincente(self, nodoCorrente, parziale, pesoCorrente, maxLength): 2 usages  ⚡ tommasogazzano
    if pesoCorrente > self._bestWeight:
        self._bestWeight = pesoCorrente
        self._bestPath = copy.deepcopy(parziale)

    if len(parziale)-1 == maxLength:
        return

    for n in self._graph.neighbors(nodoCorrente):
        if n not in parziale:
            pesoArco = self._graph[nodoCorrente][n]["weight"]
            parziale.append(n)
            self.ricorsioneCamminoVincente(n, parziale, pesoCorrente + pesoArco, maxLength)
            parziale.pop()
```

Figura 4: Algoritmo ricorsivo cammino vincente

Visualizzazione del cammino ricorsivo

Una volta terminata l’esecuzione dell’algoritmo ricorsivo, i metodi drawBestPathToFile() e drawWorstPathToFile() consentono di generare una nuova

versione dell'immagine del grafo, nella quale viene evidenziato il cammino individuato. In particolare, il percorso vincente viene rappresentato in verde, mentre quello debole in rosso, per facilitare l'interpretazione dei risultati.

```
def drawBestPathToFile(self, bestPath, outpath: str = "best_path.png") -> str: 1 usage  1 tommasogazi
    """
    Disegna il grafo complessivo, evidenziando il cammino vincente trovato con la ricorsione
    :param BestPath: lista dei nodi del cammino vincente
    :param outpath: nome file png di output
    :return: percorso assoluto del file
    """
    if self._graph.number_of_nodes() == 0:
        raise ValueError("Grafo vuoto, niente da disegnare")

    isolated_nodes = list(nx.isolates(self._graph))
    self._graph.remove_nodes_from(isolated_nodes)

    if self._graph.number_of_nodes() == 0:
        raise ValueError("Nessun nodo connesso da disegnare")

    if bestPath is None or len(bestPath) == 0:
        raise ValueError("Cammino Vuoto, niente da Evidenziare")

    pos = nx.circular_layout(self._graph)
    fig, ax = plt.subplots(figsize=(14.4, 10))

    #disegna il grafo di base
    nx.draw_networkx_nodes(self._graph, pos, ax=ax, node_size=300)
    nx.draw_networkx_edges(self._graph, pos, ax=ax, edge_color='lightgray', width=1.5)

    #evidenzia gli archi appartenenti al cammino trovato
    path_edges = []
    for i in range(len(bestPath)-1):
        u = bestPath[i]
        v = bestPath[i + 1]
        if self._graph.has_edge(u, v):
            path_edges.append((u, v))
    nx.draw_networkx_edges(self._graph, pos, path_edges, edge_color='Green', width=3)

    #evidenzia i nodi appartenenti al cammino trovato
    nx.draw_networkx_nodes(self._graph, pos, ax=ax, nodelist = bestPath, node_size=300)

    labels = {n: n.Retailer_name for n in self._graph.nodes()}
    nx.draw_networkx_labels(self._graph, pos = pos, ax=ax, labels=labels, font_size=12)

    edges_labels = nx.get_edge_attributes(self._graph, name='weight')
    nx.draw_networkx_edge_labels(self._graph, pos=pos, ax=ax, edge_labels=edges_labels, font_size=10)
    ax.set_axis_off()
    plt.margins(0.20)
    plt.savefig(*args: outpath, dpi=200, bbox_inches='tight')
    plt.close()
    return os.path.abspath(outpath)
```

Figura 5: elaborazione PNG algoritmo ricorsivo

Analisi dei prodotti lungo il cammino

In seguito all'individuazione del cammino vincente o del cammino debole, l'applicazione effettua un'analisi sui prodotti associati ai retailer presenti nei rispettivi percorsi. Per il cammino vincente vengono individuati i tre prodotti più frequenti lungo la sequenza, mentre per il cammino debole vengono estratti i tre prodotti meno ricorrenti.

Queste informazioni vengono poi mostrate insieme alla rappresentazione grafica del cammino corrispondente, permettendo di affiancare alla visualizzazione della rete anche un'indicazione sintetica dei prodotti maggiormente rappresentativi del percorso analizzato.

Diagramma delle classi dell'applicazione

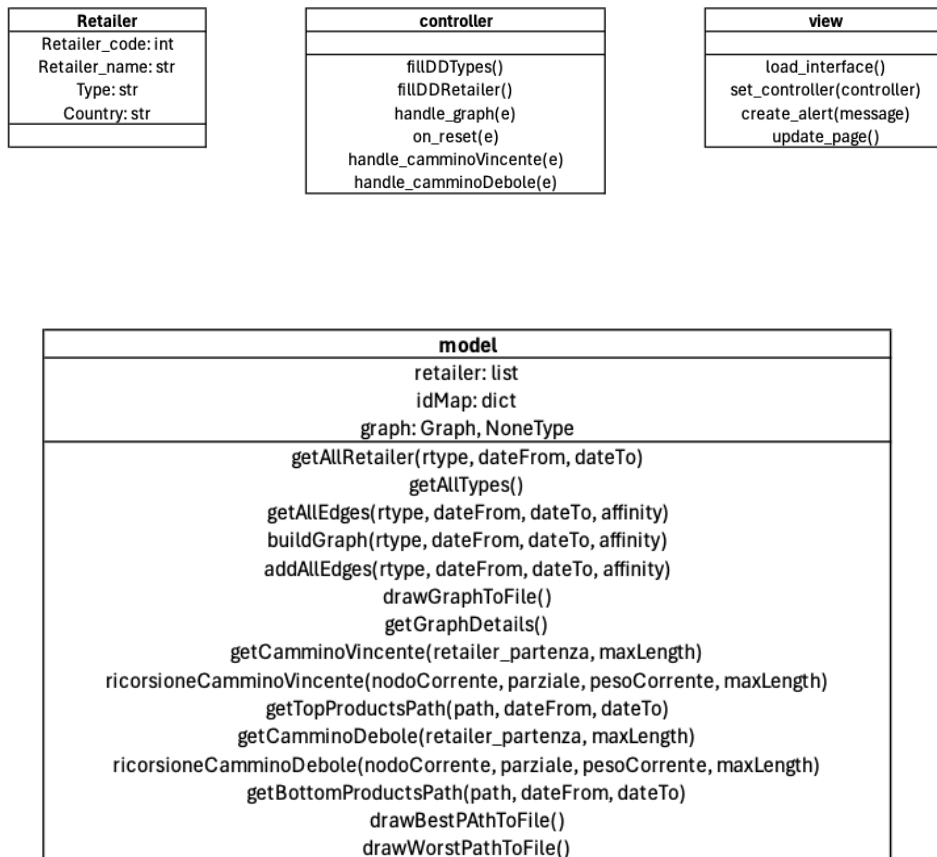


Figura 6: Diagramma classi progetto

Esempio applicativo e risultati sperimentali

Funzionamento dell'applicazione

All'avvio dell'applicazione, l'interfaccia grafica consente la selezione diversi filtri, come la tipologia di retailer, la data di inizio e di fine della ricerca e la soglia di affinità, di cui è stata inserita la definizione in un apposito container flet. Una volta che il controller ha validato i filtri senza riscontrare incongruenze, la pressione del pulsante "Crea Grafo" avvia la generazione del grafo e la sua visualizzazione. Solo in seguito a questa operazione vengono sbloccati ulteriori filtri necessari all'esecuzione degli algoritmi ricorsivi, i cui risultati vengono mostrati nell'area sottostante.

Una volta terminata la ricerca è possibile resettare tutti i parametri selezionati attraverso l'apposito pulsante "Reset Filtri" ritornando alla condizione di avvio dell'applicazione.

Di seguito viene proposta una simulazione con tipologia retailer "Warehouse Store".



Figura 7: Interfaccia grafica all'avvio

Esempio Warehouse Store

Utilizzando come simulazione la tipologia di retailer “Warehouse Store”, intervallo temporale compreso tra il 22/12/2015 – 22/12/2018, soglia di affinità uguale a 4, si ottiene il seguente grafo.

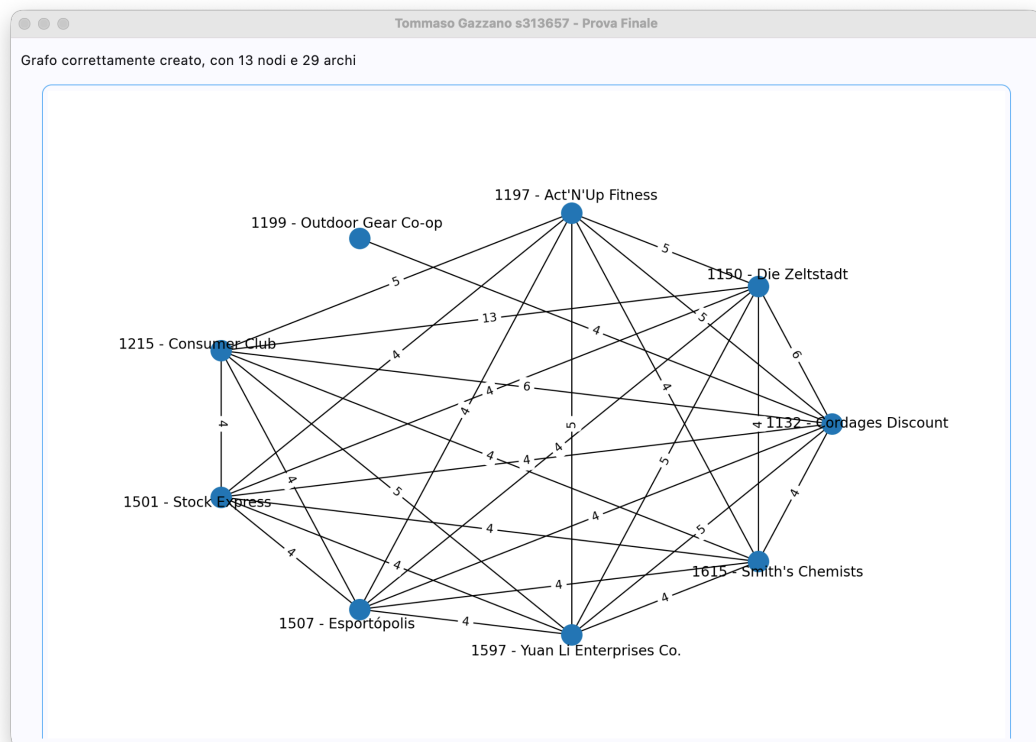


Figura 8: Creazione e visualizzazione grafo

Generato correttamente il grafo l'applicazione permette di selezionare dal menù a tendina uno dei retailer presenti e dall'apposito slider una lunghezza massima per i cammini. Andando a selezionare “Act'N'Up Fitness”, lunghezza massima pari a 5 ed eseguendo la ricerca del cammino vincente si ottengono i risultati di seguito riportati.

Attraverso questa schermata l'utente non si limita a visualizzare il percorso ottimale tra i punti vendita, evidenziato in verde, ma riceve indicazioni sui Top 3 prodotti, ovvero quelli che hanno generato il maggior numero di vendite in comune. Questo dato è utile nella gestione pratica del business: oltre a supportare la pianificazione delle scorte per evitare che i prodotti che generano più fatturato vadano esauriti, l'analisi permette di individuare quali siano gli articoli che meritano una maggiore spinta pubblicitaria.

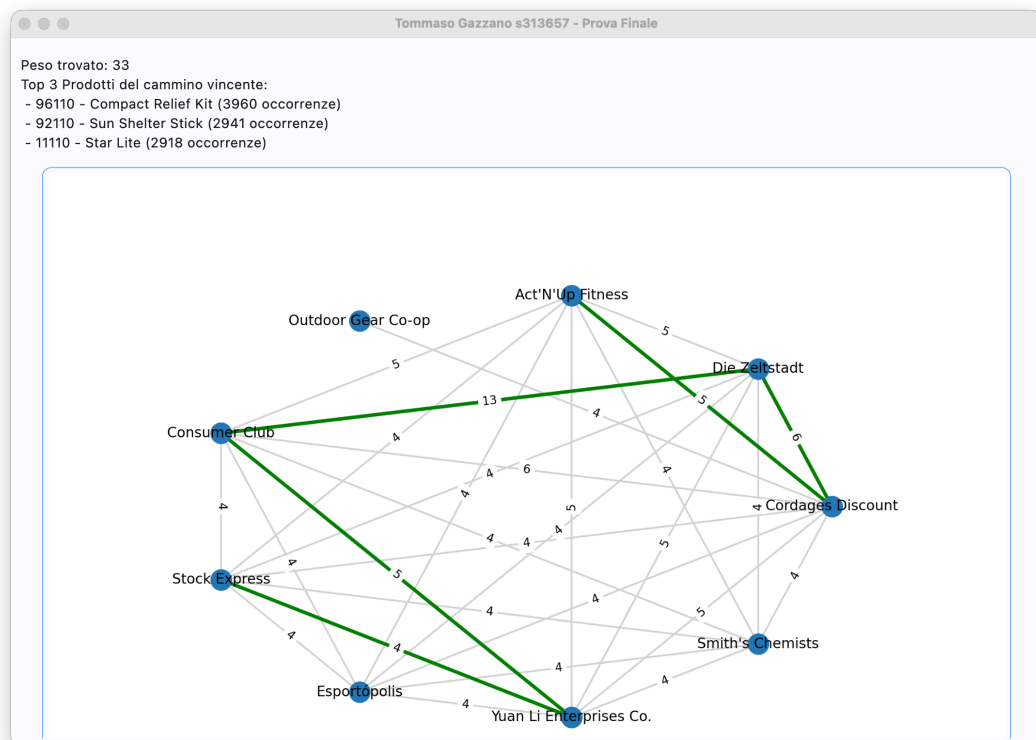


Figura 9: Algoritmo ricorsivo vincente e visualizzazione grafo

In modo analogo, l'applicazione permette di esaminare i Bottom 3 prodotti associati al cammino debole, evidenziato in rosso, identificando quegli articoli che hanno registrato il minor numero di vendite all'interno della rete selezionata. Questa analisi permette di individuare tempestivamente eventuali criticità: l'utente può infatti decidere di avviare strategie di liquidazione mirata per smaltire le giacenze rimaste invendute, oppure valutare la rimozione definitiva del prodotto dal proprio catalogo.

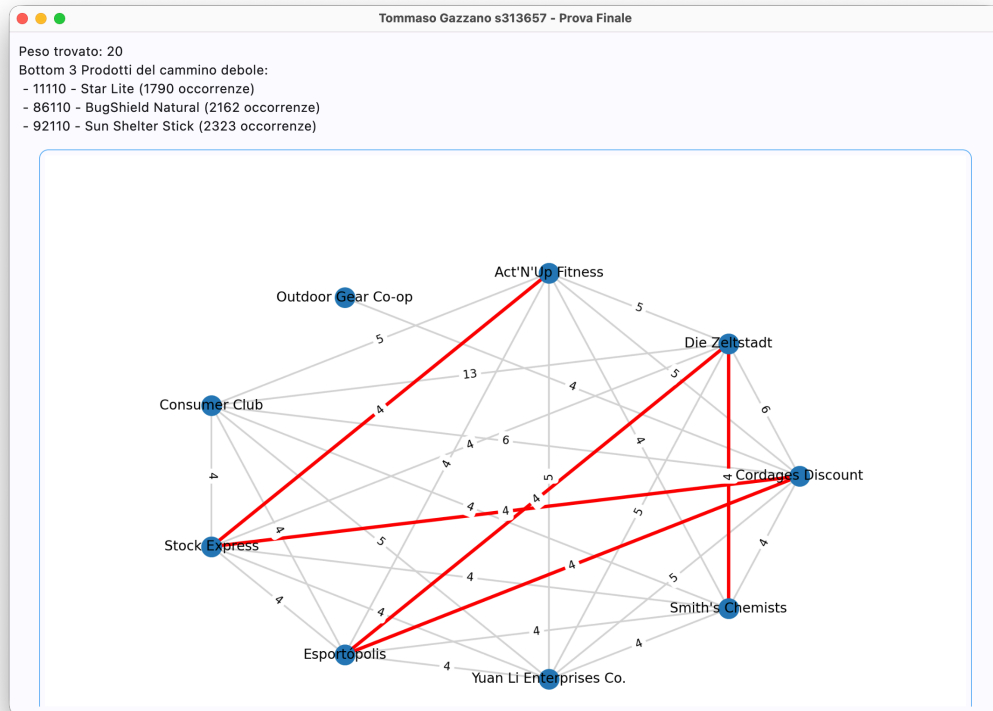


Figura 10: Algoritmo ricorsivo debole e visualizzazione grafo

Valutazioni sui risultati ottenuti e conclusioni

Limiti dell'applicazione

La complessità degli algoritmi utilizzati cresce rapidamente con l'aumentare dei nodi e delle lunghezze dei cammini, rendendo necessario imporre vincoli per mantenere i tempi di esecuzione accettabili. Inoltre, i risultati dipendono in modo significativo dai parametri scelti e dalla qualità dei dati disponibili.

In prospettiva sarebbe interessante introdurre tecniche di analisi più avanzate. In particolare l'integrazione di tecniche di machine learning potrebbe consentire di individuare automaticamente pattern nascosti nelle relazioni tra i rivenditori. Tali approcci potrebbero inoltre contribuire a migliorare l'efficienza computazionale del sistema, ad esempio guidando l'esplorazione delle soluzioni evitando quelle poco promettenti.

Valutazione del risultati

Il lavoro svolto ha mostrato come l'utilizzo di un modello a grafo possa essere efficiente per analizzare le relazioni tra rivenditori a partire dai dati di vendita. La costruzione della rete ha permesso di rappresentare in modo chiaro le connessioni tra i diversi attori. L'utilizzo di algoritmi ricorsivi ha reso possibile approfondire ulteriormente l'analisi, individuando sequenze di rivenditori caratterizzate da relazioni forti o deboli. In particolare, il "cammino vincente" ha evidenziato catene più coese all'interno della rete, mentre il "cammino debole" ha permesso di individuare situazioni meno consolidate, potenzialmente utili per valutare interventi di miglioramento.

Dal punto di vista applicativo, i risultati ottenuti possono essere utili per supportare alcune decisioni operative, quali la gestione dell'assortimento o l'individuazione di prodotti su cui investire maggiormente. L'analisi dei prodotti più frequenti e meno frequenti lungo i cammini individuati aggiunge infatti un livello di dettaglio che rende l'interpretazione dei risultati più concreta.

Licenza

Il software sviluppato è rilasciato con licenza MIT, che consente di utilizzare, copiare, modificare e ridistribuire il codice, a scopo sia personale sia commerciale, a condizione che sia mantenuto il copyright originario. Il testo completo della licenza può essere consultato online al seguente link: <https://opensource.org/license/MIT>